

Assignment 7: Build and Evaluate Unsupervised Learning Models

Juan Maldonado Franco

DDS-8555 Predictive Analysis

Mohamed Nabeel

```
In [1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path.cwd()
for parent in [ROOT, *ROOT.parents]:
    if (parent / "DDS-8555 - Predictive Analysis").exists():
        COURSE = parent / "DDS-8555 - Predictive Analysis"
        break
else:
    COURSE = ROOT.parents[1]
DATA = COURSE / "data"
KAGGLE = DATA / "kaggle"
SUBMISSIONS = DATA / "submissions"
RANDOM_STATE = 42
pd.set_option("display.max_columns", 80)
```

Conceptual Question 1

The K-means objective can be written as the sum of within-cluster squared distances. The identity in the text shows that this is equivalent to measuring pairwise distances within each cluster up to a constant. The algorithm decreases the objective at each iteration because the assignment step chooses the closest centroid for each observation, and the update step sets each centroid to the mean of its assigned observations. Each step cannot increase the within-cluster sum of squares, so the procedure moves toward a local optimum.

Applied Question 9: USArrests Hierarchical Clustering

The USArrests exercise shows why scaling matters in unsupervised learning. Without scaling, variables with larger units dominate Euclidean distance. After scaling, each variable contributes on a comparable standard-deviation scale.

```
In [2]: import statsmodels.api as sm
from scipy.cluster.hierarchy import linkage, fcluster
```

```

from sklearn.preprocessing import StandardScaler

usarrests = sm.datasets.get_rdataset("USArrests", "datasets").data
Z_raw = linkage(usarrests, method="complete", metric="euclidean")
raw_clusters = pd.Series(fcluster(Z_raw, 3, criterion="maxclust"), index=usarrests.index)
scaled = StandardScaler().fit_transform(usarrests)
Z_scaled = linkage(scaled, method="complete", metric="euclidean")
scaled_clusters = pd.Series(fcluster(Z_scaled, 3, criterion="maxclust"), index=scaled.index)
display(pd.concat([raw_clusters, scaled_clusters], axis=1).sort_values(["scaled_clusters"]))

```

	raw_cluster	scaled_cluster
rownames		
Delaware	1	1
Arkansas	2	1
Massachusetts	2	1
Missouri	2	1
New Jersey	2	1
Oklahoma	2	1
Oregon	2	1
Rhode Island	2	1
Virginia	2	1
Washington	2	1
Wyoming	2	1
Connecticut	3	1
Hawaii	3	1
Idaho	3	1
Indiana	3	1
Iowa	3	1
Kansas	3	1
Kentucky	3	1
Maine	3	1
Minnesota	3	1

Wine PCA and Clustering

The Wine clustering data set is scaled before PCA, K-means, and hierarchical clustering.

Scaling is required because the chemical measurements are on different units and would

otherwise distort distance-based methods.

```
In [3]: from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

wine = pd.read_csv(KAGGLE / "wine-dataset-for-clustering" / "wine-clustering.csv")
X_scaled = StandardScaler().fit_transform(wine)
pca_full = PCA().fit(X_scaled)
cumulative = np.cumsum(pca_full.explained_variance_ratio_)
n_components = int(np.argmax(cumulative >= .80) + 1)
pca = PCA(n_components=n_components, random_state=RANDOM_STATE)
scores = pca.fit_transform(X_scaled)
print("Components needed for 80% variance:", n_components)
display(pd.DataFrame({"component": range(1, len(cumulative)+1), "cumulative_variance": cumulative}))
rows = []
for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=20).fit(scores)
    rows.append({"k": k, "inertia": km.inertia_, "silhouette": silhouette_score(scores, km.labels_)})
display(pd.DataFrame(rows))
hc = AgglomerativeClustering(n_clusters=3, linkage="ward").fit(scores)
pd.Series(hc.labels_, name="hierarchical_cluster").value_counts().sort_index()
```

Components needed for 80% variance: 5

	component	cumulative_variance
0	1	0.361988
1	2	0.554063
2	3	0.665300
3	4	0.735990
4	5	0.801623
5	6	0.850981
6	7	0.893368
7	8	0.920175
8	9	0.942397
9	10	0.961697

	k	inertia	silhouette
0	2	1201.157500	0.323917
1	3	825.020838	0.369076
2	4	723.935749	0.325619
3	5	661.429070	0.307802
4	6	605.376056	0.272779
5	7	558.113912	0.275598

```
Out[3]: hierarchical_cluster
0      55
1      58
2      65
Name: count, dtype: int64
```

Interpretation

PCA retained at least 80% of the variance with fewer components than the original feature set, which makes the clustering task easier to visualize and less redundant (Jolliffe, 2002). K-means was evaluated across several k values rather than assuming a single cluster count. Hierarchical clustering gave a second view of the grouping structure and helped test whether the K-means result was a method artifact.

References

Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer.

<https://doi.org/10.1007/b98835>

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281-297).

Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301), 236-244.

<https://doi.org/10.1080/01621459.1963.10500845>